Lesson 13 Teacher's Key: A Hit That Reacts

GameMaker - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Introducing Alarms - GameMaker's built-in 'do something after N steps' mechanism - genuinely new this week, and necessary because GML can't pause mid-event the way Godot's await or Unity's Coroutine can. This requires splitting the hit-handling logic across TWO events for the first time.

Level 1 - Full Worked Solution

- obj_bullet's Collision event keeps ALL of its existing logic (sound, score, win-check) completely unchanged and immediate - only instance_destroy(other) is replaced with a flash + an alarm.
- obj_enemy's new Alarm 0 event is deliberately tiny - just instance_destroy() - because all the meaningful game-state logic already ran in the Collision event, 6 steps earlier.
- obj_explosion is a fully separate object with its own Create-event alarm and its own Alarm 0 event - it manages its own lifetime with zero involvement from obj_enemy or obj_bullet.

Why This Structure

- Splitting into two events (Collision starts the flash + alarm, Alarm 0 finishes the removal) is the idiomatic GameMaker way to express 'do this, then after a pause, do that' - GML doesn't support blocking waits inside a single event the way Godot or Unity's languages do.
- Keeping the win-check/score logic in the Collision event (unchanged) rather than moving it into the Alarm 0 event keeps this week's change small and localized - GameMaker students are learning ONE new concept (Alarms), not restructuring everything they already built.
- The flash swaps sprite_index rather than tinting image_blend white, for the same reason Lesson 2's Recolor swaps sprites instead of tinting them: image_blend multiplies against the sprite's existing color, and multiplying any color by white is a no-op - our enemy sprite is opaque red, so a white image_blend would produce zero visible change.

Level 2 / Level 3 Talking Points

- Ask students to trace exactly which lines moved and which didn't between Lesson 12 and this week - only instance_destroy(other) changed location (into a new event); everything else is untouched.

- For students comparing engines: point out that GameMaker's alarm[0] + Alarm 0 event is conceptually identical to TIC-80's flash-countdown-in-a-table approach this week - both engines lack a blocking wait, so both split 'trigger' and 'finish' into two separate places.

Common Student Mistakes

- Leaving instance_destroy(other) in place alongside the new alarm code, causing the enemy to still vanish instantly (the alarm gets set but never has a chance to fire).
- Forgetting to add the new Alarm 0 event to obj_enemy at all - setting alarm[0] does nothing without a matching Alarm event to catch it.
- Accidentally moving score/win-check logic INTO the new Alarm 0 event instead of leaving it in the Collision event - this would delay scoring and win detection by 6 steps, which is unnecessary and inconsistent with every other engine this week.